Roteiro da apresentação final — Memphis-V Graphical Debugger

Nov 12, 2026 · @Guilherme Rosmann

Os quatro integrantes apresentam em sequência, cerca de 2,5 a 3,5 minutos cada, somando 11 min 10 s e deixando 50 s de folga nos 12 minutos. Slides: `documentation/Apresentacao_Final_Memphis-V_Debugger.pptx`.

Divisão

Ordem	Quem	Slides	Assunto	Tempo
1	Max Weiss Breuel	1 a 4	Abertura, objetivo, hipóteses e método	3:20
2	Guilherme de Lima Rosmann	5 e 6	O que foi entregue e H1 (fidelidade)	2:40
3	Matheus Lopes Azambuja	7 e 8	H2 (desempenho) e H3 (métricas do artigo)	2:35
4	Laura Barcelos Rosa	9 a 11	H4, falhas, próximos passos e encerramento	2:35

A divisão é uma sugestão: os blocos têm tempos parecidos e podem trocar de dono. O Max, responsável pelas apresentações, abre e explica o método, que inclui o pipeline de release feito por ele. Todos ajudaram a reescrever o código, então qualquer um pode ficar com os slides de resultados.

Cada um termina passando a palavra para o próximo pelo nome (frase em itálico no fim de cada bloco). Quem passa o slide é quem está falando.

Max — slides 1 a 4 (3:20)

Slide 1 · Capa (0:20)

"Boa tarde. Somos Guilherme, Matheus, Laura e eu, Max, com orientação do professor Fernando Moraes. No pitch apresentamos a proposta de reescrever em Python o depurador gráfico da Memphis-V. Hoje mostramos o que entregamos, como validamos contra o original em Java e qual é o próximo passo."

Slide 2 · Objetivo (0:50)

"A Memphis-V simula chips com vários processadores ligados por uma rede-em-chip, e o depurador reproduz, pacote a pacote, o que a simulação gravou. O original é em Java e não acompanhou a pesquisa. Refizemos em Python com PySide6, classe a classe, com três metas: interface moderna, código fácil de manter e execução sem JVM. Todos os nossos testes usam este cenário de exemplo: 9 processadores, 3.101 pacotes e 11 tarefas."

- Apontar: o mapa de calor à direita (cada célula é um roteador; azul é pouco tráfego).

Slide 3 · Hipóteses (1:00)

"Registramos quatro hipóteses no caderno de pesquisa. H1: o port dá os mesmos resultados do Java. H2: mesmo sendo uma linguagem mais lenta, o Python lê o trace tão rápido quanto o Java. H3: as métricas que o artigo da Memphis promete para o depurador saem dos arquivos que já lemos. H4: dá para simplificar o código sem perder nada. A pergunta de escala do pitch continua sendo a direção da pesquisa, e o benchmark da H2 é o primeiro número que temos sobre ela."

- Não adiantar os resultados: cada hipótese tem o seu slide.

Slide 4 · Método e linha do tempo (1:10)

"Trabalhamos em ciclos curtos. Cada tarefa, uma função do Java ou uma ideia do artigo, era implementada com o Claude Code como assistente e revisada por nós, testada sem tela com o cenário de exemplo, integrada por pull request com o CI rodando os testes e gerando o executável, e registrada no caderno com data, decisões e falhas. A parte mais intensa foi em 24 e 25 de setembro: simulação, versão alpha, versão 1.0 e os experimentos. Em 13 de outubro eu coloquei no CI a geração automática dos executáveis de Linux e Windows a cada tag."

- Apontar: os dois marcos em laranja (24/09 e 25/09) e o último, 13/10.

"Agora o Guilherme mostra o que a versão 1.0 entrega."

Guilherme — slides 5 e 6 (2:40)

Slide 5 · O que a v1.0.0 entrega (1:20)

"Esta é a janela principal. A malha de roteadores agora é desenhada em vetor, com zoom; os periféricos ficam fora da malha, o caminho do pacote aparece em vermelho e cada enlace mostra o seu uso. Tudo o que o original faz está aqui: simulação passo a passo, as visões gerais, o Message Log, o Deloream, a janela do roteador, filtros e projetos. E acrescentamos o que o artigo pedia e o original não tinha: estatísticas de tráfego, linha do

tempo das aplicações, o caminho de qualquer pacote, exportação e acompanhamento ao vivo. São 42 commits, 8 pull requests, 58 testes e 21 diferenças em relação ao original, todas documentadas."

- Apontar: as setas vermelhas na captura (o pacote atual) e a linha de números embaixo.

Slide 6 · H1: os números batem com o Java (1:20)

"Primeiro resultado. Rodando o mesmo cenário nos dois, vimos setas vermelhas diferentes no final. Para saber se o erro era nosso, emulamos a lógica do Java pacote a pacote, inclusive com o float de 32 bits, nos 3.101 pacotes. Os números são idênticos: os mesmos 101 recálculos, o mesmo tempo final, os mesmos percentuais. Só as setas diferem, e a causa é um bug do original: o contador de pacotes em trânsito chega a 1.356 e nunca zera, então o Java acumula até 106 setas ao mesmo tempo. A emulação previu exatamente as duas setas que sobram no print do Java. H1 confirmada."

- Apontar: a terceira linha da tabela (em laranja) e depois os dois números grandes.

"O Matheus mostra agora o desempenho e as métricas do artigo."

Matheus — slides 7 e 8 (2:35)

Slide 7 · H2: Python lê o trace tão rápido quanto Java (0:55)

"Segundo resultado. Montamos um trace com o exemplo repetido 300 vezes, 930 mil pacotes, e medimos só a leitura e o processamento, sem interface. Java: 7,18 segundos. Python: 7,07. Empate: o Python é mais lento como linguagem, mas o código original é pouco eficiente. Do lado da interface, a versão Python foi feita para ser mais leve, com QTimer, redesenho só quando chega pacote e lotes no modo rápido, mas isso ainda não foi medido."

- Deixar claro o limite (frase em laranja): a interface não entrou na medição.

Slide 8 · H3: métricas do artigo a partir dos arquivos (1:40)

"Terceiro resultado, o mais ligado ao artigo da Memphis. A seção sobre o depurador promete filtrar por aplicação, validar o roteamento e medir latência e vazão. Implementamos tudo a partir dos arquivos que já líamos. No exemplo, nenhuma violação do roteamento XY em 1.607 registros, e a admissão de cada aplicação leva de 62 a 108 mil ciclos. Aqui também há um experimento que falhou: a primeira latência casava pacotes errados, porque o trace muda os campos de tarefa entre os hops. Seguindo cada pacote roteador a roteador, todas as 1.388 mensagens passaram a ser entregues e a mediana caiu

de 560 para 7 ciclos. A hipótese vale só em parte: a linha do tempo precisa do log do mapper, porque a Memphis-V não envia pela rede as mensagens do protocolo descrito no artigo."

- Apontar: a linha do tempo (laranja = mapeamento, roxo = alocação, azul = execução) e depois a tabela da latência.
- Se o tempo apertar, cortar a frase sobre a admissão; manter a correção da latência.

"A Laura fecha com o que aprendemos e o próximo passo."

Laura — slides 9 a 11 (2:35)

Slide 9 · H4 e o que aprendemos com as falhas (1:00)

"A quarta hipótese era sobre manutenção: juntamos módulos e passamos de 31 para 21 arquivos, sem mudar lógica, com os 69 casos de teste passando antes e depois. E, como o caderno de pesquisa pede, registramos também o que deu errado: códigos de serviço lidos em hexadecimal, a latência que casava pacotes errados, o executável aberto da pasta errada e o menu global do GNOME que trocava o nome do programa por causa do diálogo de arquivos. Cada falha está no caderno com a causa e a correção."

Slide 10 · Próximo passo: fazer o debugger escalar (1:10)

"A pergunta do pitch continua aberta. Já sabemos que 930 mil pacotes são lidos em cerca de 7 segundos, mas todos ficam na memória, e voltar no tempo reprocessa o trace desde o início. O banco de dados do artigo fica no simulador, e a Memphis-V grava texto. Então o plano segue as três etapas do pitch: armazenar num cache indexado, consultar só o que está visível e, antes de tudo, medir com um trace de cerca de 2 milhões de pacotes. Se a medição mostrar problema, o cache compensa; se não, não. Também falta abrir um debug pela linha de comando e medir o desenho da interface."

- Enfatizar "medir antes de decidir": é a resposta honesta à pergunta do pitch.

Slide 11 · Encerramento (0:25)

"Resumindo: entregamos um depurador que já substitui o original, com os mesmos resultados, os bugs corrigidos, o mesmo desempenho de leitura e as métricas que o artigo pedia. O código, a documentação e o caderno de pesquisa estão no GitHub. Obrigado, ficamos à disposição para perguntas."

- Deixar o slide 11 na tela durante as perguntas; o slide 12 (referências) só se pedirem.

Perguntas prováveis

Cada pergunta tem um responsável, para ninguém responder por cima de outro.

Pergunta	Resposta curta	Quem responde
Por que não fizeram o banco de dados do pitch?	Medir vem antes: com o exemplo tudo roda instantâneo, e um cache só compensa com traces grandes. O próximo passo é medir com ~2 milhões de pacotes.	Laura
Como sabem que os resultados são iguais aos do Java?	Emulação da lógica Java pacote a pacote, com float de 32 bits: recálculos, tempo e percentuais idênticos nos 3.101 pacotes.	Guilherme
Qual foi o papel do Claude Code?	Assistente de programação nas 9 sessões de desenvolvimento; toda mudança foi revisada, testada com o cenário de exemplo e passou pelo CI antes de entrar na `main`.	Guilherme
O benchmark de 930 mil pacotes é representativo?	É o trace do exemplo repetido 300 vezes: mede a leitura e o processamento, não a interface nem cenários com outro perfil de tráfego.	Matheus
Funciona no Windows?	Sim: desde 13/10 o CI gera os executáveis de Linux e Windows a cada tag.	Max
Por que a latência mediana é só 7 ciclos?	A maioria das mensagens do exemplo é pequena e de poucos hops; a máxima, 13.483 ciclos, é o código de uma tarefa com 2.908 flits.	Matheus
O que acontece com malhas maiores (4×4, 6×6)?	A grade e os cálculos usam as dimensões do `platform.cfg`; testamos uma malha 4×4 com clusters 2×2. Um testcase 6×6 foi criado, mas ainda não aberto no debugger.	Max

Pergunta	Resposta curta	Quem responde
Por que o Back To não avança no tempo?	Mesmo comportamento do original: só volta a tempos já lidos. Para avançar, usa-se o play em velocidade máxima.	Guilherme

Checklist antes de apresentar

- [ ] Ensaiar uma vez inteira com cronômetro: cada um dentro do seu tempo, total abaixo de 11:30.
- [ ] Combinar quem passa os slides (sugestão: quem está falando) e testar o passador ou o teclado.
- [ ] Abrir o `.pptx` no computador da apresentação e conferir se a fonte Arial e as capturas aparecem certas.
- [ ] Cada um ler os números do seu bloco em voz alta uma vez (1.356, 106, 7,18 contra 7,07, 1.607, 62 a 108 mil, 31 para 21).
- [ ] Ter o caderno de pesquisa em PDF aberto, para mostrar se perguntarem sobre o registro do trabalho.
- [ ] Ter o GitHub do projeto aberto numa aba, caso peçam para ver o código ou as releases.